Operating System Involvement with Paging

1 Core Idea

Main Idea

Paging is supported by hardware, but the operating system still has important work to do at process creation, execution, page fault handling, and process termination.

The MMU performs address translation during normal execution. The OS sets up the structures that make this translation possible and handles exceptional cases such as page faults.

2 Four Times the OS Is Involved

Time	OS paging work
Process creation	Create and initialize paging data structures.
Process execution	Make the process's page table current and reset MMU/TLB state.
Page fault	Locate the missing page, find a frame, load the page, and restart the instruction.
Process termination	Release page tables, physical pages, and disk backing space.

3 Process Creation Time

When a new process is created, the OS prepares its virtual-memory environment.

1. Determine the initial size of the program text and data.
2. Allocate memory for the process's page table.
3. Initialize the page-table entries.
4. Allocate swap-space or backing-store space on disk.
5. Initialize the backing store with program text and data if needed.
6. Record page-table and swap-area information in the process table.

Page Table Residency

The page table does not have to stay in RAM while the process is swapped out. It must be in memory when the process is running.

4 Executable File as Backing Store

Some systems do not copy program text into the swap area at process creation time.

Optimization

Program text can be paged directly from the executable file.

This saves:

- disk space, because the text pages do not need a duplicate swap copy;
- initialization time, because the OS does not need to copy them into swap space first.

5 Process Execution Time

When the scheduler chooses a process to run, the OS must make that process's address space active.

Action	Purpose
Set page-table register or pointer	Tell the MMU which page table belongs to the new process.
Flush the TLB	Remove cached translations from the previous process.
Reset MMU state	Ensure translations use the new process's mappings.
Optional prepaging	Bring likely-needed pages into memory before execution continues.

Why Flush the TLB?

The TLB may contain virtual-to-physical translations from the previously running process. If they are reused incorrectly, the new process could access the wrong physical frames.

6 Optional Prepaging

The OS may bring in some pages before the process actually faults on them.

Example

The page containing the instruction pointed to by the program counter will certainly be needed, so the OS may load it early.

Prepaging can reduce the initial burst of page faults when a process begins or resumes.

7 Page Fault Time

When a page fault occurs, the OS handles the missing-page event.

1. Read hardware registers to find the faulting virtual address.
2. Determine which virtual page contains that address.
3. Locate the needed page on disk.

4. Find a free page frame.
5. If no free frame exists, choose a victim page to evict.
6. If the victim is dirty, write it back to disk.
7. Read the needed page into the selected frame.
8. Update the page table and relevant TLB information.
9. Back up or reset the program counter to the faulting instruction.
10. Restart the instruction.

Page Fault Flow

faulting address → needed page → free frame → load page → restart instruction

8 Why the Instruction Is Restarted

The faulting instruction did not complete successfully because the page was missing.

Restart Rule

After the page has been loaded, the CPU must execute the same instruction again, now with the needed page present.

This is why the OS may need to back up the program counter or otherwise restore the faulting instruction's state.

9 Process Termination Time

When a process exits, the OS must clean up its paging resources.

Resource	Cleanup action
Page table	Release the memory used for the process's page table.
Physical pages	Free the frames owned only by this process.
Swap space	Release backing-store disk space.
Shared pages	Release only if no other process still uses them.

Shared Page Warning

If a page is shared, the OS must not free its memory or disk backing store until the last process using it has terminated or unmapped it.

10 Lifecycle Summary

Stage	Main OS job	Key data structures
Creation	Build address-space metadata.	Page table, process table, swap map.
Execution	Activate correct mappings.	MMU registers, TLB, page-table pointer.
Page fault	Bring missing page into RAM.	Page table, frame table, disk backing store.
Termination	Free resources safely.	Page table, frame table, swap map, shared-page metadata.

11 Final Summary

One-Box Summary

At creation, the OS builds page tables and backing-store metadata.
At execution, it loads the MMU state and flushes stale TLB entries.
At a page fault, it finds, loads, maps, and retries the missing page.
At termination, it releases page tables, frames, and disk space safely.